

# Project Requirement Development Fun Run Management System

## 1. Metadata and Overview

### Basic Information

- **Document Title:** Product Requirement Development: Fun Run Management System (Operational Planning and Event Coordination)
- **Author:** Abdul Aliff Ammar Bin Abdul Khalid (Student)
- **Supervisor:** Dr. Nazean Binti Jomhari
- **Academic Year:** 2026
- **Version History:** v1.0 (Initial Draft focusing on Core Infrastructure: Modules 1–6)
- **Key Stakeholders:** Public User / Participants, Event Organizers, System Administrators

### Executive Summary

Fun runs attract large, diverse crowds of various ages and fitness levels, making operational planning surprisingly complex. Fragmented management platforms often lead to registration bottlenecks, communication breakdowns, and administrative overhead.

The **Fun Run Management System** is an integrated web-based platform designed to automate the entire event lifecycle. While the ultimate system encompasses end-to-end race coordination, this initial development cycle focuses on building the foundational pipeline—spanning **Module 1 (User Management)** through **Module 6 (E-Ticket Module)**. This creates a secure, seamless workflow from user onboarding and event discovery to registration, payment, and digital ticket issuance.

## 2. The "Why" (Objectives and Context)

### Problem Statement

Current industry platforms fail to provide a cohesive operational solution for organizers:

- **Fragmented Pre-Event Lifecycle:** Organizers must frequently patch together different software tools to handle user profiles, event listings, custom registration forms, and secure payment tallies.
- **Lack of Direct Communication & Ticket Tracking:** Competitor systems (e.g., RaceXAsia, JomRun) often lack centralized, automated tools to connect registration records directly with verifiable digital ticketing formats prior to race day.

## Business Goals

- **Centralize the Fun Run Ecosystem:** Provide a single, unified interface that minimizes administrative data handling.
- **Automate Credential Issuance:** Shift 100% of the registration-to-ticketing pipeline to self-service automation, eliminating manual data entry.

## Success Metrics

- **Reduced Time-to-Register:** Lower average registration and payment completion time to under 3 minutes per user.
- **Zero Discrepancy Ticketing:** Ensure a 100% accurate 1:1 match between successfully processed payments and unique generated E-Tickets.

## Background / Context

Popular platforms like HeyJom, RaceXAsia, and JomRun leave critical feature gaps in integrated operations. For instance, JomRun lacks dedicated operational check-in tools, and RaceXAsia offers minimal automated communication channels. This project builds a localized, scalable framework to fill these gaps, informed by first-hand volunteer observations at community events like the *Larian Kesedaran Autisme*.

## 3. Target Audience

### User Personas

1. **The Public User / Participant:** A recreational runner seeking quick event discovery, straightforward team/individual registration, and instant digital ticket delivery.
2. **The Event Organizer:** An administrator needing an intuitive dashboard to create events, configure categories (e.g., distances, pricing), and track registration tallies.
3. **The System Admin:** A high-level technical operator responsible for platform governance, managing user privileges, and overseeing general operational reports.

### Use Cases

- **UC5 & UC6 (Event Selection to Ticket Pipeline):** A Public User browses available events, fills in registration details, completes payment, and instantly receives an electronic ticket.
- **UC9 (Event Management):** An Event Organizer configures specific run requirements (shirt sizes, emergency contacts) and publishes the event live.

## 4. Features & Requirements

### User Stories

- **As a Participant**, I want to browse fun run events by location and category, so I can find runs that match my fitness level and schedule.
- **As a Participant**, I want to register and securely pay online, so I can instantly secure my slot and receive a digital confirmation.
- **As an Event Organizer**, I want to create a centralized event posting, so that I can capture all participant metrics (shirt sizes, categories) automatically.

### Functional Requirements

| FR ID | Module              | System Requirement Description                                                                                                    | Primary User        |
|-------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------|
| FR-01 | User Management     | System must handle account registration, secure login, profile modification, and role assignment (Participant, Organizer, Admin). | All Users           |
| FR-02 | Event Management    | Allows authorized creators to add, edit, or delete event parameters (date, venue, runner categories, descriptions).               | Admin / Organizer   |
| FR-03 | Event Discovery     | Provides public search inputs and dynamic filters based on location and category parameters.                                      | Public User         |
| FR-04 | Registration Module | Captures detailed participant inputs (shirt_size, emergency_contact, selected tier) for individual or team entries.               | Participant         |
| FR-05 | Payment Module      | Processes transaction amounts, logs dates, and updates states (payment_status = 'Success/Failed') securely.                       | Participant         |
| FR-06 | E-Ticket Module     | Generates a distinct digital voucher embedding a unique QR code tied to the registration_id upon successful payment.              | Participant / Admin |

## Acceptance Criteria

### Scenario: Successful Registration and E-Ticket Generation

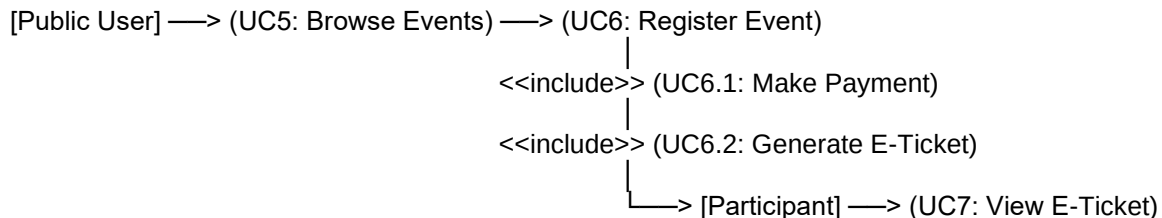
- **Given** a registered participant is on the event checkout screen for an approved fun run;
- **When** they submit valid payment details through the payment module and the transaction is approved;
- **Then** the system should update the database record status to "Paid", instantiate a unique record in the E-Ticket database, generate a verifiable QR code, and display the ticket immediately for download.

### Scope Boundary

- **In-Scope (Current Phase):** Core registration engine containing Modules 1, 2, 3, 4, 5, and 6.
- **Out-of-Scope (Future Phase):** Real-time race day execution components, specifically: Module 7 (Check-in scanning), Module 8 (Automated notifications), Module 9 (Post-event report exporter), Module 10 (Automated certificates), and Module 11 (Sponsor suggestions).

## 5. Design and User Flow

### Visual Context



The entity relationships mapping the structural tables (USER, EVENT, REGISTRATION, PAYMENT, and E\_TICKET) are carefully documented in the system's Entity-Relationship Diagram to maintain relational database integrity.

## 6. System & Technical Requirements

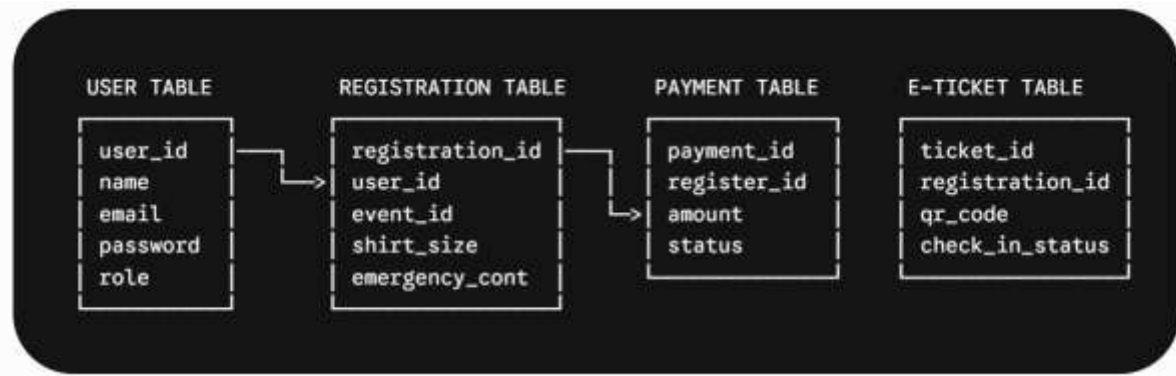
### Non-Functional Requirements (NFRs)

According to technical baselines established during the project's monitoring phase:

- **Performance (NFR03):** Pages, search criteria updates, and transaction confirmation windows must fully load within 3 seconds under standard internet conditions.
- **Security (NFR07):** Endpoints handling sensitive actions (editing profiles, managing tickets) must enforce session token validation via secure user authentication.

- **Reliability (NFR05):** Registration and transactional records must guarantee ACID compliance during concurrent writes to prevent double-booking or missing ticket allocations.

## Technical Specifications



## 7. Constraints, Risks, & Dependencies

### Constraints

- **Development Model:** System architecture adheres strictly to a sequential Waterfall Model, meaning phase transitions depend on finalizing static documentation prior to programming.

### Dependencies

- **Payment Gateway Mocking:** Since this phase directly introduces the Payment Module (Module 5), the execution loop depends on a reliable Sandbox API environment to safely simulate transaction behaviors. Just mock up a payment gateway instead of creating an actual payment module that needed to pay before register.
- **QR Engine Library:** Module 6 requires dependency integration with a stable QR generation library to dynamically translate relational string hashes into renderable image matrix blocks.

### Open Questions

- Will organizers require custom fields per event tier (e.g., collecting unique dietary needs for specific long-distance categories) within Module 4, or will standard attributes suffice?

## 8. Release Plan & Timeline

Following the structured phase framework defined in the development of Modules 1 through 6 adheres to targeted milestones:

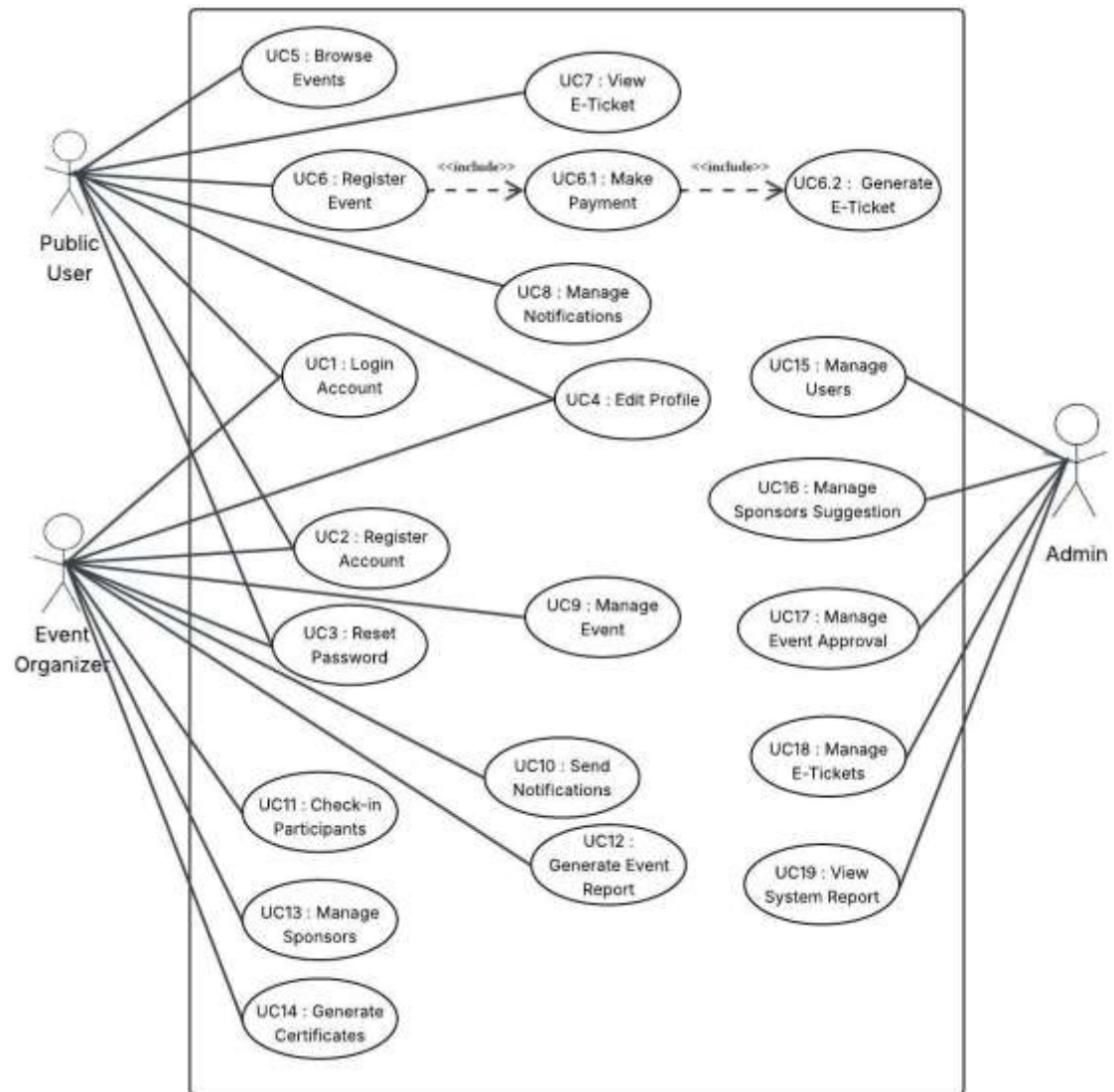
Phase 1: Requirements —> Finalizing current operational scope definitions.

Phase 2: Design —> Reviewing ERD schemas and system entity mappings[cite: 1].

Phase 3: Development —> Programming sprint covering Modules 1 through 6.

Phase 4: Testing —> Executing functional tests and User Acceptance Testing (UAT)[cite: 1]  
Deployment into production will serve as the stable technical baseline before initiating Phase 2 development (Modules 7–11)[cite: 2].

## 9. Use Case Diagram



## 10. Modules

| No. | Module Name         | Description                                        | Attributes (Data Fields)                                                              | Users              |
|-----|---------------------|----------------------------------------------------|---------------------------------------------------------------------------------------|--------------------|
| 1   | User Management     | Manage user accounts (register, login, profile)    | user_id, name, email, password, phone_no, role                                        | Participant, Admin |
| 2   | Event Management    | Create and manage fun run events                   | event_id, event_name, date, location, category, description, organizer_id             | Admin              |
| 3   | Event Discovery     | Users browse/search events                         | search_keyword, category_filter, location_filter, event_list                          | Participant        |
| 4   | Registration Module | Participants register for events (individual/team) | registration_id, user_id, event_id, category, shirt_size, emergency_contact           | Participant        |
| 5   | Payment Module      | Handle online payment & transaction                | payment_id, registration_id, amount, payment_method, payment_status, transaction_date | Participant        |
| 6   | E-Ticket Module     | Generate digital ticket after registration         | ticket_id, registration_id, QR_code, check_in_status                                  | Participant, Admin |



# 11. Frameworks

## 1. Backend: Laravel (PHP)

Laravel is widely considered the most AI-friendly backend framework in existence because its syntax reads like plain English.

- **Built-in authentication:** You can generate a complete, secure User Management module (Module 1) including login, registration, and password resets in literally **one command** using *Laravel Breeze* or *Jetstream*.
- **Industry Relevance:** As seen in your competitive analysis, industry leaders like **HeyJom** rely on Laravel to run their core systems.

## 2. Frontend: Blade Templates + Tailwind CSS

- **Why it's great for vibe coding:** Blade files blend standard HTML with simple backend logic. The AI can write your frontend UI, loop through your "Event Discovery" database entries (Module 3), and format it using **Tailwind CSS** utility classes all within a single file.
- It completely eliminates the need to build a separate REST API just to get your frontend to talk to your backend.

## 3. Database: SQLite (for Development) —> MySQL (for Final Submission)

- **Why it's great for vibe coding:** Laravel supports SQLite out of the box. It requires zero configuration, zero local server installations (like XAMPP database troubleshooting), and stores your entire database inside a single local file. When you are ready to submit your project, changing it to **MySQL** (which matches your system's design requirements) takes exactly one line of code modification in your environment configuration file.